

Rmq

RMQ

简介

RMQ 是英文 Range Maximum/Minimum Query 的缩写，表示区间最大（最小）值。

在接下来的描述中，默认初始数组大小为 n ，询问次数为 m 。

在接下来的描述中，默认时间复杂度标记方式为 $O(n)$ ，其中 $O(n)$ 表示预处理时间复杂度，而 $O(1)$ 表示单次询问的时间复杂度。

单调栈

由于 OI Wiki 中已有此部分的描述，本文仅给出 [链接](#)。这部分不再展开。

时间复杂度 $O(n)$ ，空间复杂度 $O(n)$ 。

ST 表

由于 OI Wiki 中已有此部分的描述，本文仅给出 [链接](#)。这部分不再展开。

时间复杂度 $O(n \log n)$ ，空间复杂度 $O(n \log n)$ 。

线段树

由于 OI Wiki 中已有此部分的描述，本文仅给出 [链接](#)。这部分不再展开。

时间复杂度 $O(n \log n)$ ，空间复杂度 $O(n \log n)$ 。

Four Russian

Four russian 是一个由四位俄罗斯籍的计算机科学家提出来的基于 ST 表的算法。

在 ST 表的基础上 Four russian 算法对其做出的改进是序列分块。

具体来说，我们将原数组——我们将其称之为数组 A ——每 $\sqrt{n}$ 个分成一块，总共 $\sqrt{n}$ 块。

对于每一块我们预处理出来块内元素的最小值，建立一个长度为 $\sqrt{n}$ 的数组 B ，并对数组 B 采用 ST 表的方式预处理。

同时，我们对于数组 A 的每一个零散块也建立一个 ST 表。

询问的时候，我们可以将询问区间划分为不超过 1 个数组 B 上的连续块区间和不超过 2 个数组 A 上的整块内的连续区间。显然这些问题我们通过 ST 表上的区间查询解决。

在 $O(1)$ 时候，预处理复杂度达到最优，为 $O(n \sqrt{n})$ 。

时间复杂度 $O(n \sqrt{n})$ ，空间复杂度 $O(n \sqrt{n})$ 。

当然询问由于要跑三个 ST 表，该实现方法的常数较大。

- 一些小小的算法改进
- 一些玄学的算法改进

加减 1RMQ

若序列满足相邻两元素相差为 1，在这个序列上做 RMQ 可以成为加减 1RMQ，根究这个特性可以改进 Four Russian 算法，做到 的时间复杂度， 的空间复杂度。

由于 Four russian 算法的瓶颈在于块内 RMQ 问题，我们重点去讨论块内 RMQ 问题的优化。

由于相邻两个数字的差值为 1，所以在固定左端点数字时 长度不超过 的右侧序列种类数为 1，而这个式子显然不超过 。

这启示我们可以预处理所有不超过 种情况的 最小值 - 第一个元素 的值。

在预处理的时候我们需要去预处理同一块内相邻两个数字之间的差，并且使用二进制将其表示出来。

在询问的时候我们找到询问区间对应的二进制表示，查表得出答案。

这样子 Four russian 预处理的时间复杂度就被优化到了 。

笛卡尔树在 RMQ 上的应用

不了解笛卡尔树的朋友请移步 [笛卡尔树](#)。

不难发现，原序列上两个点之间的 min/max，等于笛卡尔树上两个点的 LCA 的权值。根据这一点就可以借助 求解树上两个点之间的 LCA 进而求解 RMQ。树上 LCA 在 [LCA - 标准 RMQ](#) 已经有描述，这里不再展开。

总结一下，笛卡尔树在 RMQ 上的应用，就是通过将普通 RMQ 问题转化为 LCA 问题，进而转化为加减 1 RMQ 问题进行求解，时间复杂度为 $O(n \log n)$ 。当然由于转化步数较多，RMQ 常数较大。

如果数据随机，还可以暴力在笛卡尔树上查找。此时的时间复杂度为期望 $O(n)$ ，并且实际使用时这种算法的常数往往很小。

例题 [Luogu P3865【模板】ST 表](#)

基于状压的线性 RMQ 算法

隐性要求

- 序列的长度 满足 $n \leq 10^5$ 。

前置知识

- [Sparse Table](#)
- 基本位运算
- 前后缀极值

算法原理

将原序列 分成每块长度为 的 块。

听说令块长为 时常数较小。

记录每块的最大值，并用 ST 表维护块间最大值，复杂度 $O(n \log n)$ 。

记录块中每个位置的前、后缀最大值（即 到其所在块的块首的最大值），复杂度 $O(n)$ 。

若查询的 在两个不同块上，分别记为第 块，则最大值为 块间的最大值，以及 和 这三个数的较大值。

现在的问题在于若 在同一块中怎么办。

将 依次插入单调栈中，记录下标和值，满足值从栈底到栈顶递减，则 中的最大值为从栈底往上，单调栈中第一个满足其下标 的值。

由于 是 中的最大值，因而在插入 时， 都被弹出，且在插入 时不可能将 弹出。

而如果用 表示每个数是否在栈中，就可以用整数状压，则 为第 位后的第一个 的位置。

由于块大小为 ，因而最多不超过 位，可以用一个整数存下（即隐性条件的原因）。

► 参考代码

习题

[\[BJOI 2020\] 封印](#): SAM+RMQ

本页面最近更新：2024/10/9 22:38:42, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [StudyingFather](#), [Back11ght](#), [billchenchina](#), [Chrogeek](#), [countercurrent-time](#), [diauweb](#), [Entertainer](#), [Henry-ZHR](#), [hsfzLZH1](#), [lr1d](#), [kfy666](#), [ksyx](#), [Mooos-MoSheng](#), [orzAtalod](#), [ouuan](#), [ranwen](#), [SkqLiao](#), [sshwy](#), [Tiphereth-A](#), [zhouyuyang2002](#), [zzjbb](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用